Summary of Float Self-Tagging Artifact Results

Contents

1 Overview

This document summarizes all results from the Float Self-Tagging artifact. It includes figures that were omitted in the Float Self-Tagging paper to avoid redundancy and save space.

Some experiments were not executed on some machines, either because the proper tooling does not exists (such as branch prediction on an Apple M2) or because the experiment takes too long to run (such as the heap experiment on RISC-V, which would take several weeks).

1.1 Adding a Machine to this Document

To include results from a new machine in this summary:

1. generate figures with `scripts/plot.sh`.[1]
2. update the \`machineslist` macro in `summary.tex`.
3. regenerate this pdf from `summary.tex`.

This document was generated for the following machines:

AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0
Has memory profiling: **true**
Has branch prediction profiling: **true**
Has preloaded heap experiment: **true**

Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0
Has memory profiling: **true**
Has branch prediction profiling: **true**
Has preloaded heap experiment: **true**

Apple M2 Max, 64GB, macOS 15.4.1, gcc 14.3.0
Has memory profiling: **false**
Has branch prediction profiling: **false**
Has preloaded heap experiment: **true**

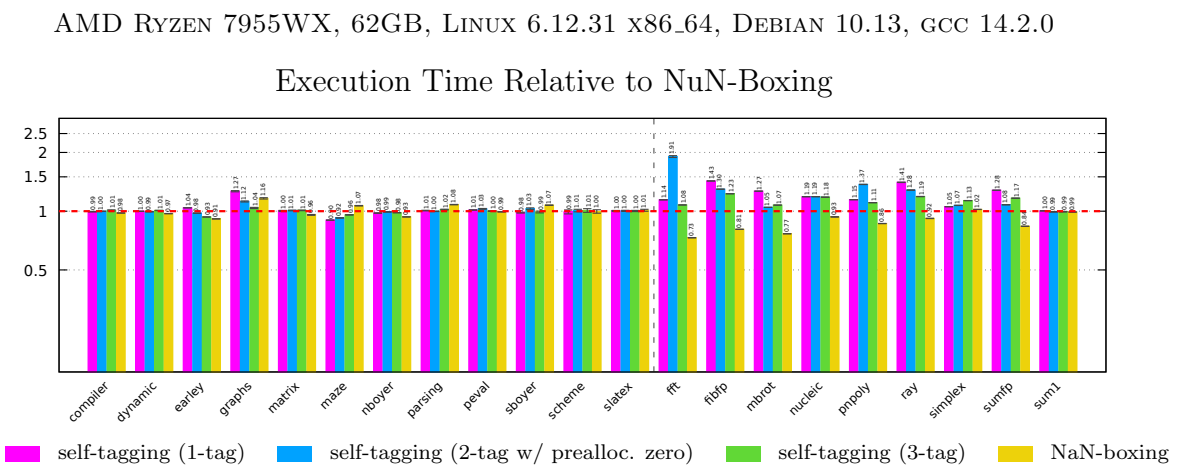
riscv sifive, u74-mc, 8GB, Linux starfive 6.1.31-starfive riscv64, gcc 14.2.0
Has memory profiling: **false**
Has branch prediction profiling: **false**
Has preloaded heap experiment: **false**

[1] to generate plots for another machine use `FST_HOST=[name] scripts/plot.sh`, where `[name]` is the output of `uname -n` on the target machine.

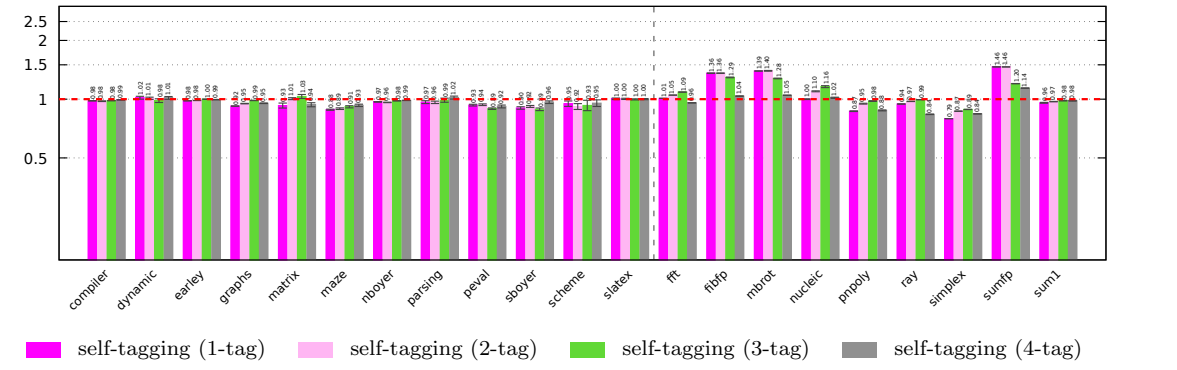
2 Results

2.1 AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

2.1.1 Execution Time Relative to NuN-Boxing



(a) Bigloo

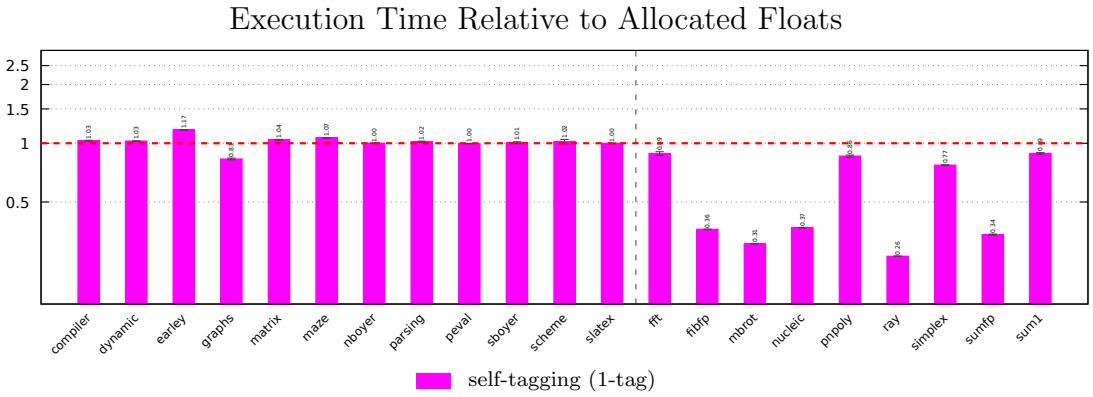


(b) Gambit

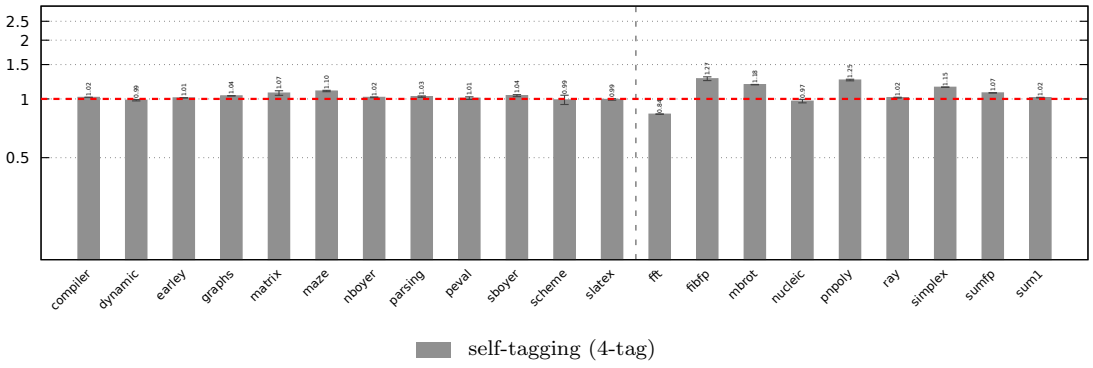
Figure 1: Execution time of self-tagging variants and NaN-boxing. The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

2.1.2 Execution Time Relative to Allocated Floats

AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0



(a) Bigloo



(b) Gambit

Figure 2: Execution time relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

2.1.3 Execution Time With Non-Empty Heap

AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

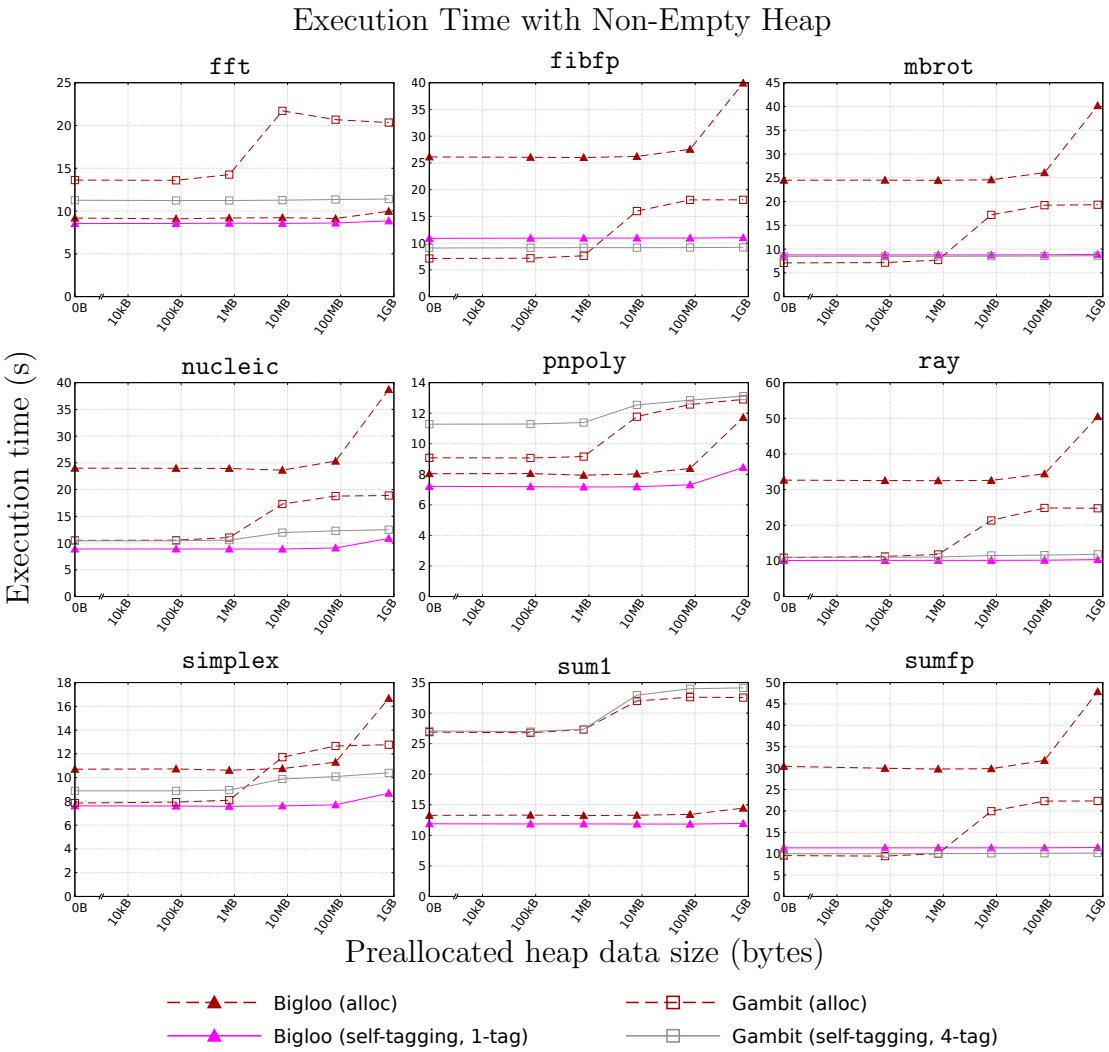


Figure 3: Execution time of float benchmarks with extra data allocated on heap before execution with Bigloo and Gambit. The x-axis shows the size of preallocated data in bytes on a logarithmic scale. The y-axis represents execution time in seconds. Execution time without preallocated data is added at $x = 0$. Dashed lines correspond to times with allocated floats. Solid lines are with self-tagging.

2.1.4 Memory Profiling

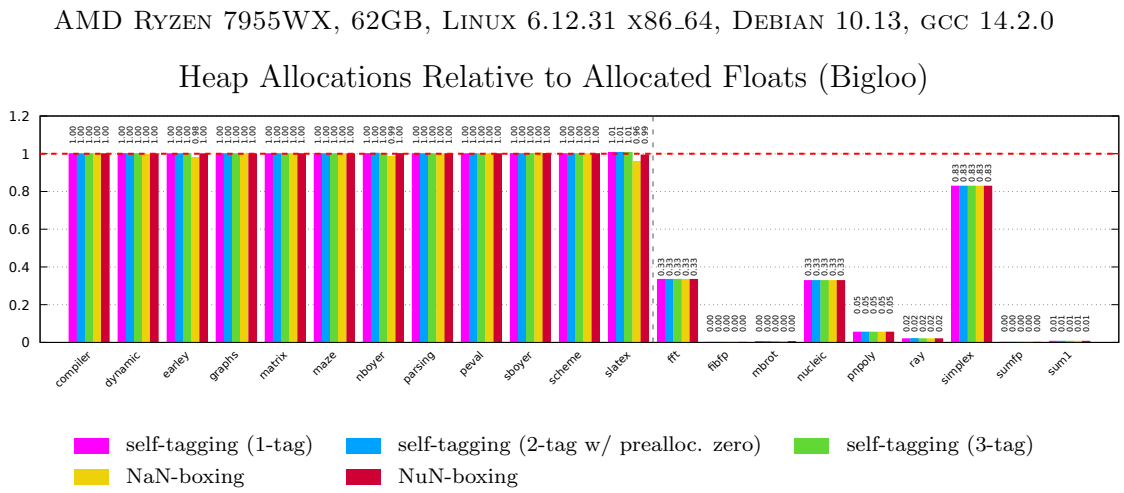


Figure 4: Heap allocations of Bigloo with each variant of self-tagging, NaN-boxing, NuN-boxing, and allocated floats. The y-axis shows the number of allocated bytes relative to the version that uses tagged pointers for all floats (dashed horizontal red line). 1.00× indicates no change, while 0.00× indicates no heap allocation.

2.1.5 Branch Misprediction

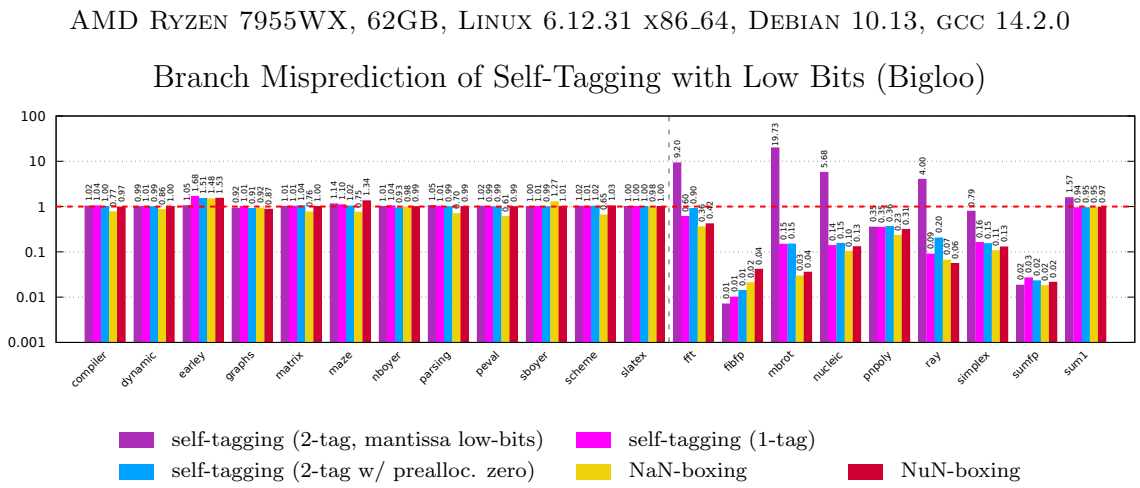


Figure 5: Branch mispredictions of Bigloo with each variant of self-tagging, NaN-boxing, and NuN-boxing relative to allocated floats. The y-axis shows branch mispredictions relative to allocated floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means less, and higher means more mispredictions.

2.1.6 Execution Time of Self-Tagging with Mantissa Low Bits

AMD Ryzen 7955WX, 62GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

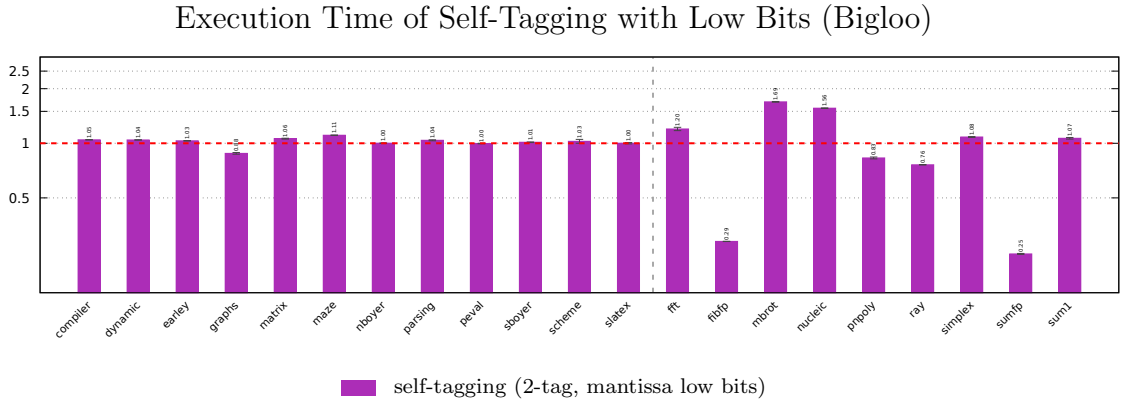
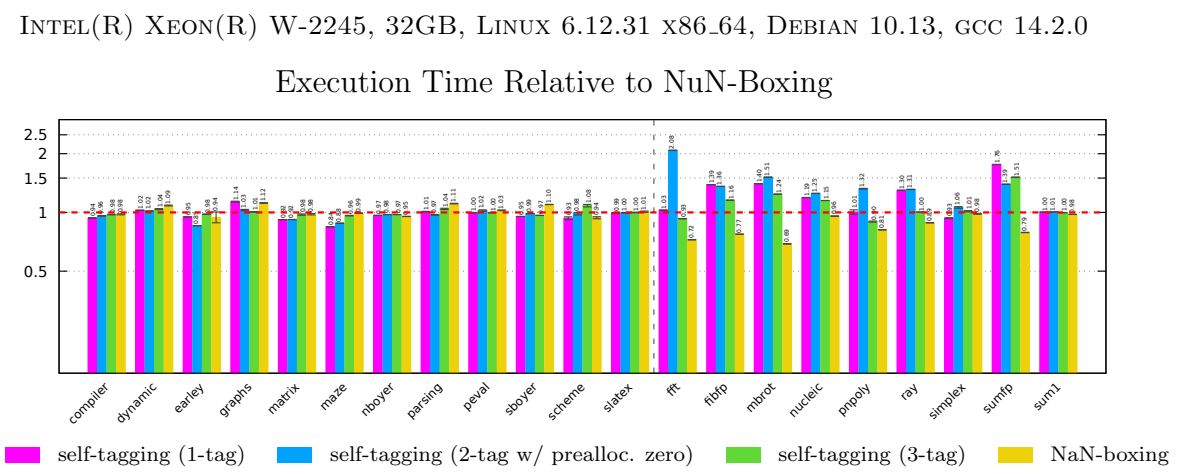


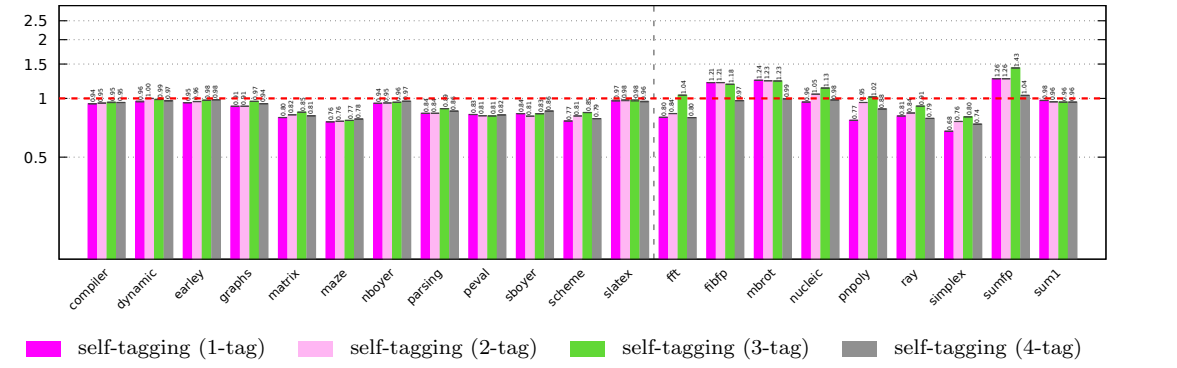
Figure 6: Execution time of Bigloo with self-tagging using the mantissa low bits relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than allocated floats.

2.2 Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

2.2.1 Execution Time Relative to NuN-Boxing



(a) Bigloo

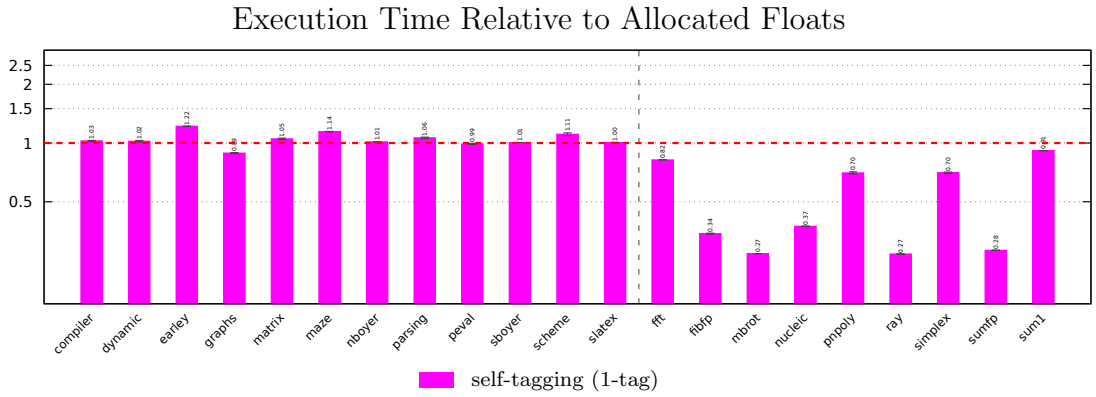


(b) Gambit

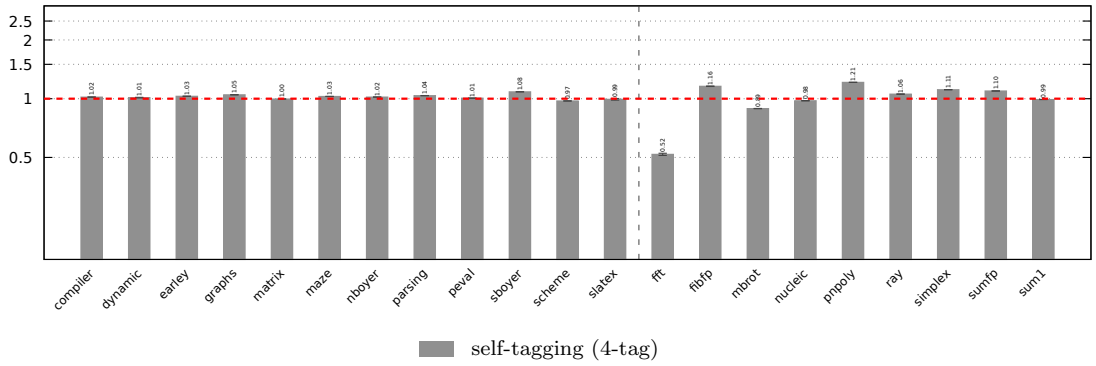
Figure 7: Execution time of self-tagging variants and NaN-boxing. The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

2.2.2 Execution Time Relative to Allocated Floats

Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0



(a) Bigloo



(b) Gambit

Figure 8: Execution time relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

2.2.3 Execution Time With Non-Empty Heap

Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

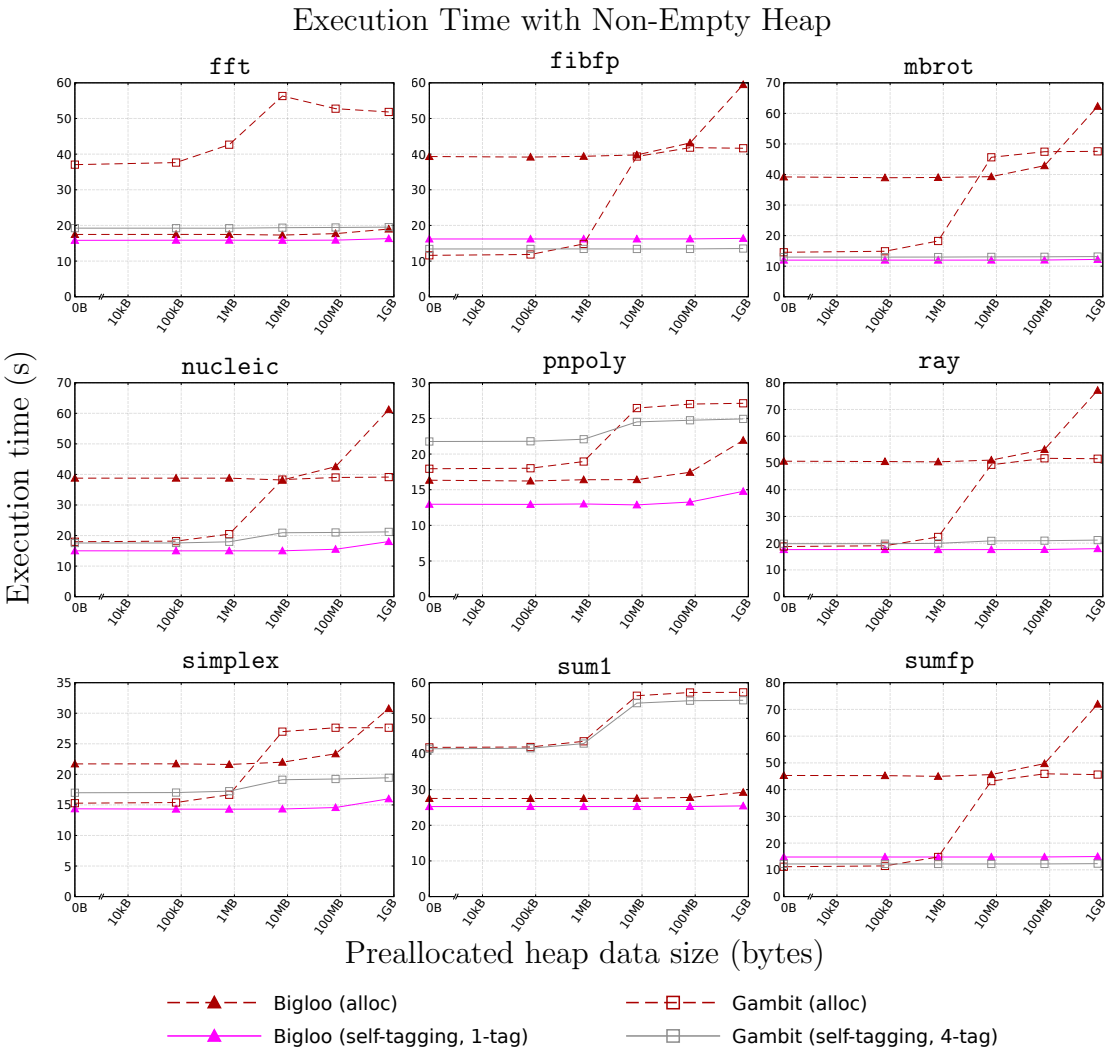


Figure 9: Execution time of float benchmarks with extra data allocated on heap before execution with Bigloo and Gambit. The x-axis shows the size of preallocated data in bytes on a logarithmic scale. The y-axis represents execution time in seconds. Execution time without preallocated data is added at $x = 0$. Dashed lines correspond to times with allocated floats. Solid lines are with self-tagging.

2.2.4 Memory Profiling

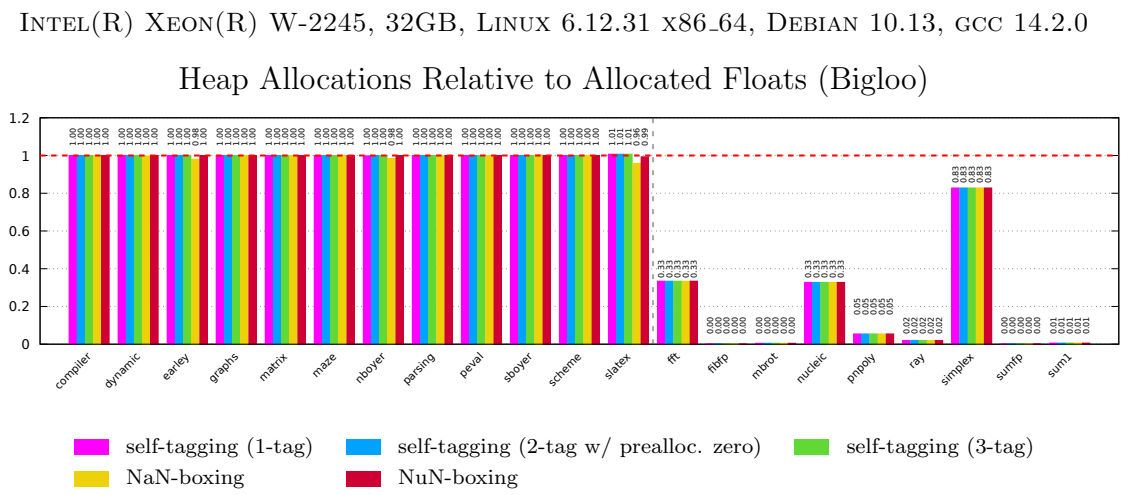


Figure 10: Heap allocations of Bigloo with each variant of self-tagging, NaN-boxing, NuN-boxing, and allocated floats. The y-axis shows the number of allocated bytes relative to the version that uses tagged pointers for all floats (dashed horizontal red line). 1.00× indicates no change, while 0.00× indicates no heap allocation.

2.2.5 Branch Misprediction

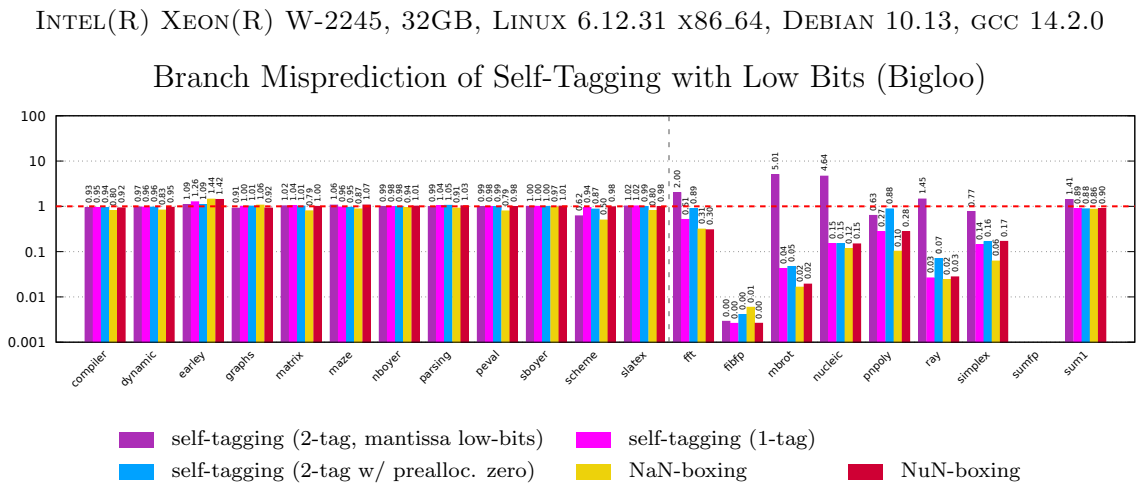


Figure 11: Branch mispredictions of Bigloo with each variant of self-tagging, NaN-boxing, and NuN-boxing relative to allocated floats. The y-axis shows branch mispredictions relative to allocated floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means less, and higher means more mispredictions.

2.2.6 Execution Time of Self-Tagging with Mantissa Low Bits

Intel(R) Xeon(R) W-2245, 32GB, Linux 6.12.31 x86_64, Debian 10.13, gcc 14.2.0

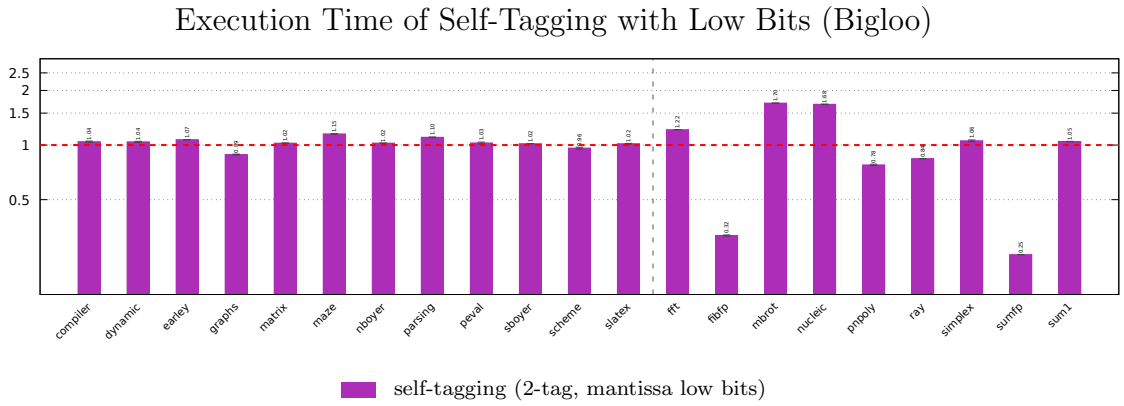
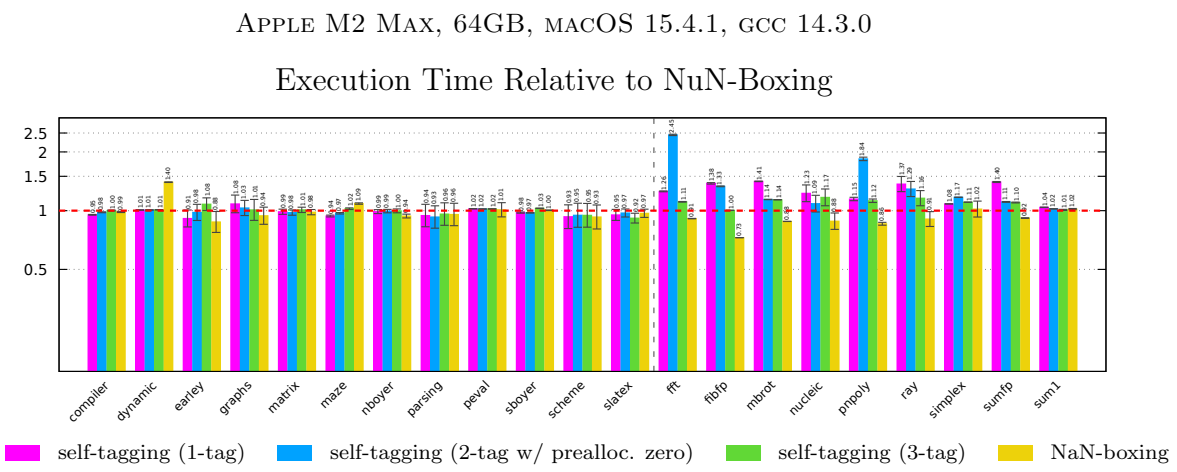


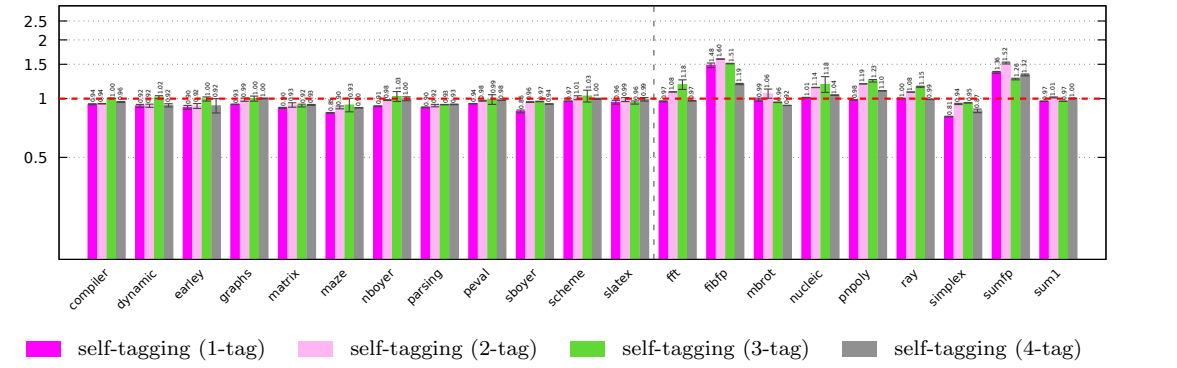
Figure 12: Execution time of Bigloo with self-tagging using the mantissa low bits relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than allocated floats.

2.3 Apple M2 Max, 64GB, macOS 15.4.1, gcc 14.3.0

2.3.1 Execution Time Relative to NuN-Boxing



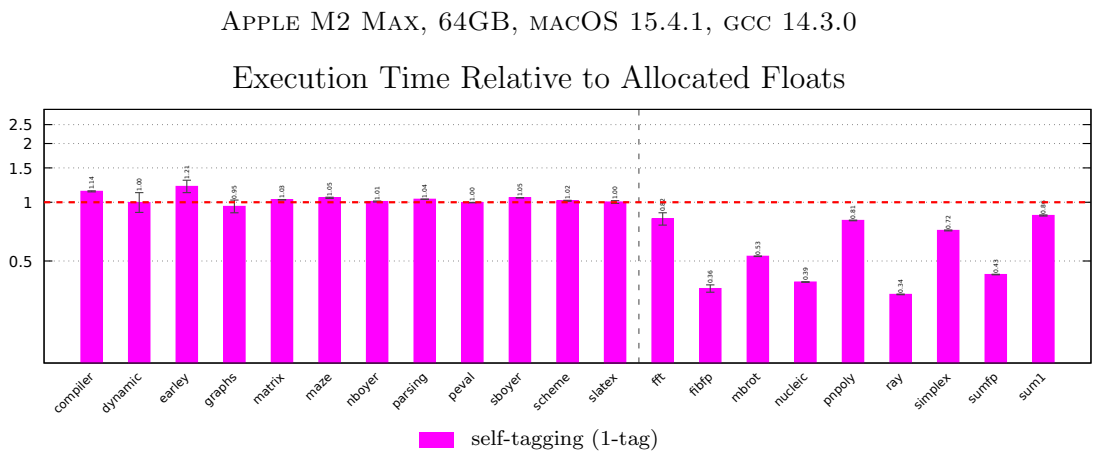
(a) Bigloo



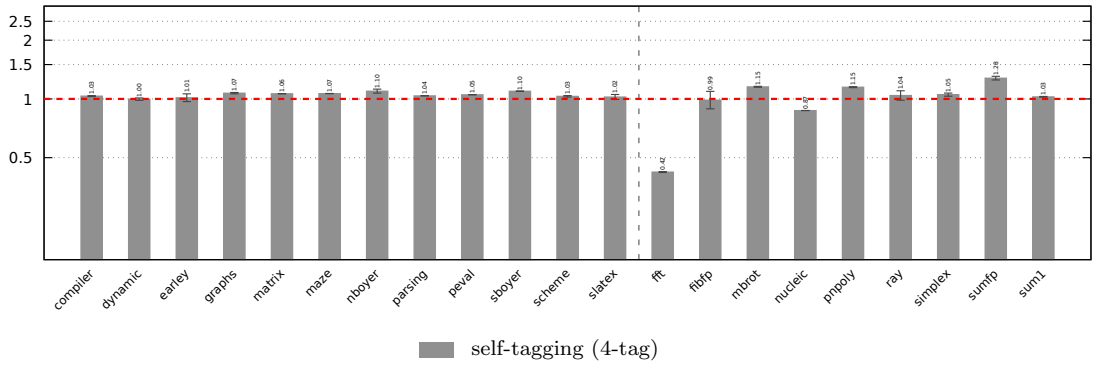
(b) Gambit

Figure 13: Execution time of self-tagging variants and NaN-boxing. The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

2.3.2 Execution Time Relative to Allocated Floats



(a) Bigloo



(b) Gambit

Figure 14: Execution time relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

2.3.3 Execution Time With Non-Empty Heap

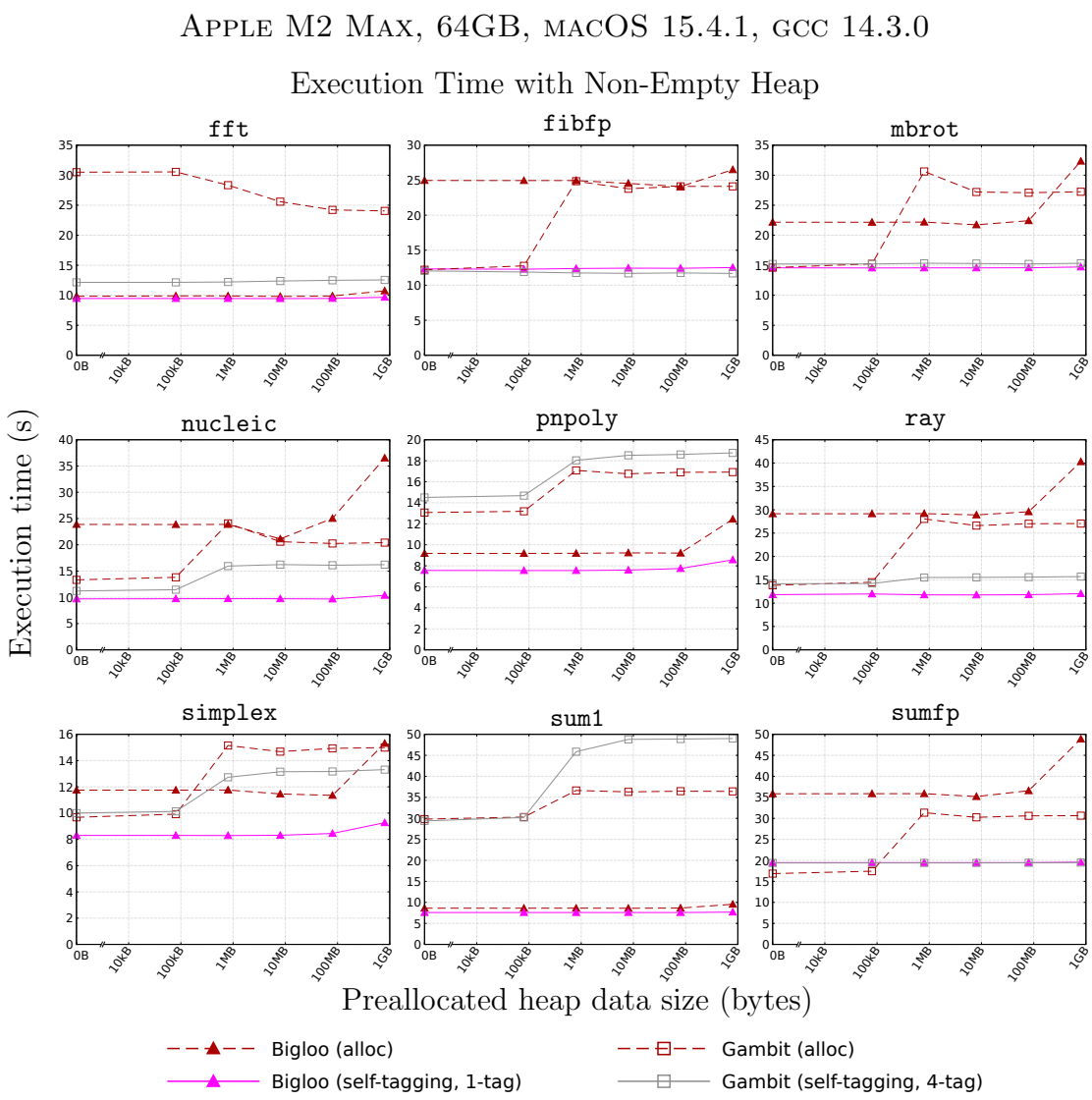


Figure 15: Execution time of float benchmarks with extra data allocated on heap before execution with Bigloo and Gambit. The x-axis shows the size of preallocated data in bytes on a logarithmic scale. The y-axis represents execution time in seconds. Execution time without preallocated data is added at $x = 0$. Dashed lines correspond to times with allocated floats. Solid lines are with self-tagging.

2.3.4 Execution Time of Self-Tagging with Mantissa Low Bits

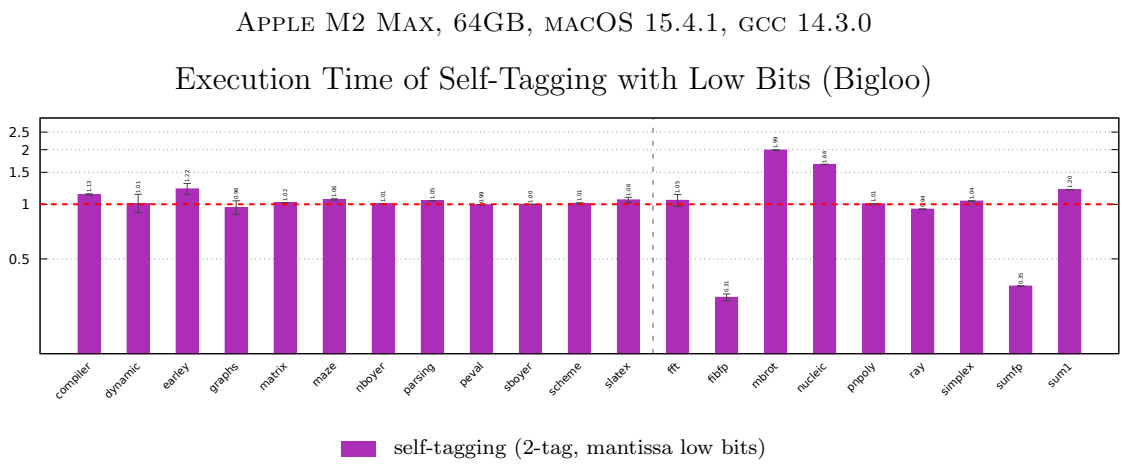
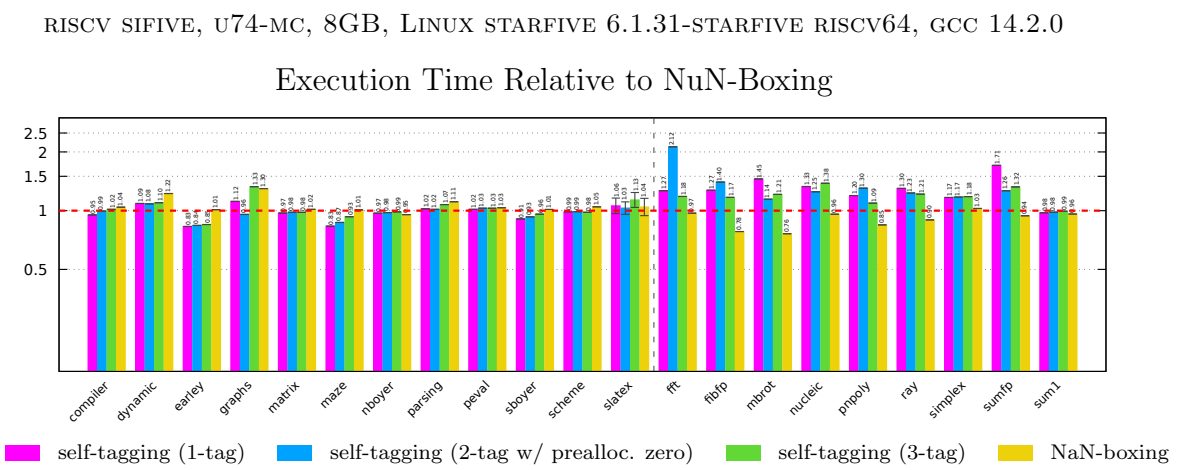


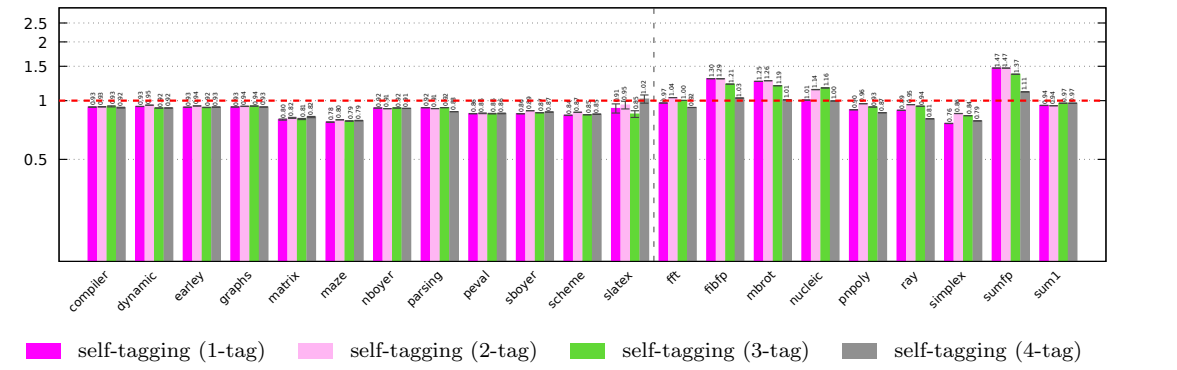
Figure 16: Execution time of Bigloo with self-tagging using the mantissa low bits relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than allocated floats.

2.4 riscv sifive, u74-mc, 8GB, Linux starfive 6.1.31-starfive riscv64, gcc 14.2.0

2.4.1 Execution Time Relative to NuN-Boxing



(a) Bigloo

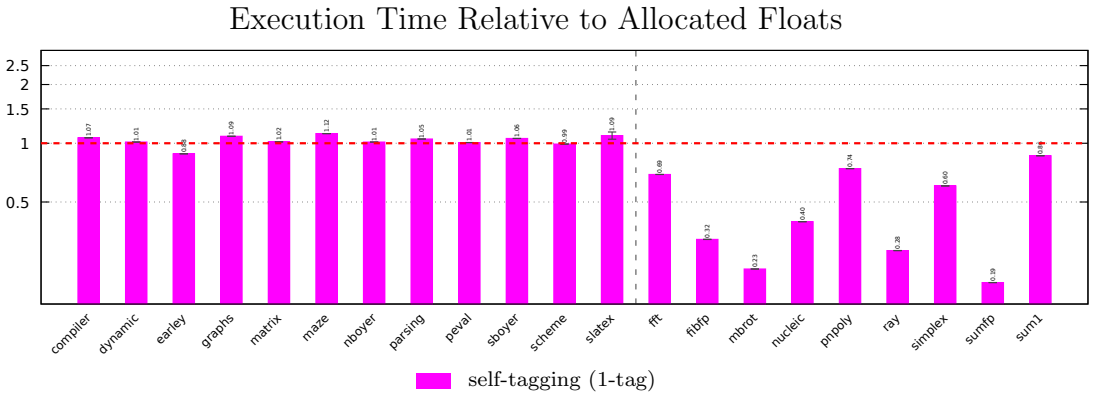


(b) Gambit

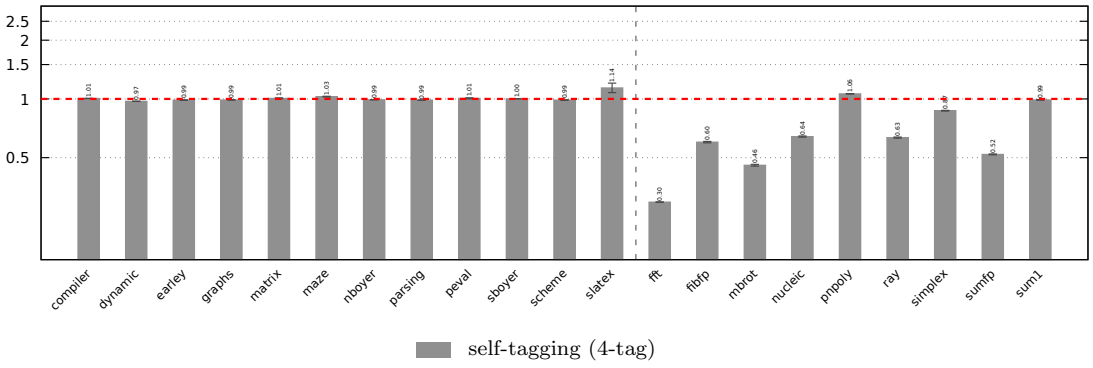
Figure 17: Execution time of self-tagging variants and NaN-boxing. The y-axis shows execution time relative to NuN-boxing (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than NuN-boxing.

2.4.2 Execution Time Relative to Allocated Floats

RISCV SIFIVE, U74-MC, 8GB, LINUX STARFIVE 6.1.31-STARFIVE RISCV64, GCC 14.2.0



(a) Bigloo



(b) Gambit

Figure 18: Execution time relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster execution than allocated floats.

2.4.3 Execution Time of Self-Tagging with Mantissa Low Bits

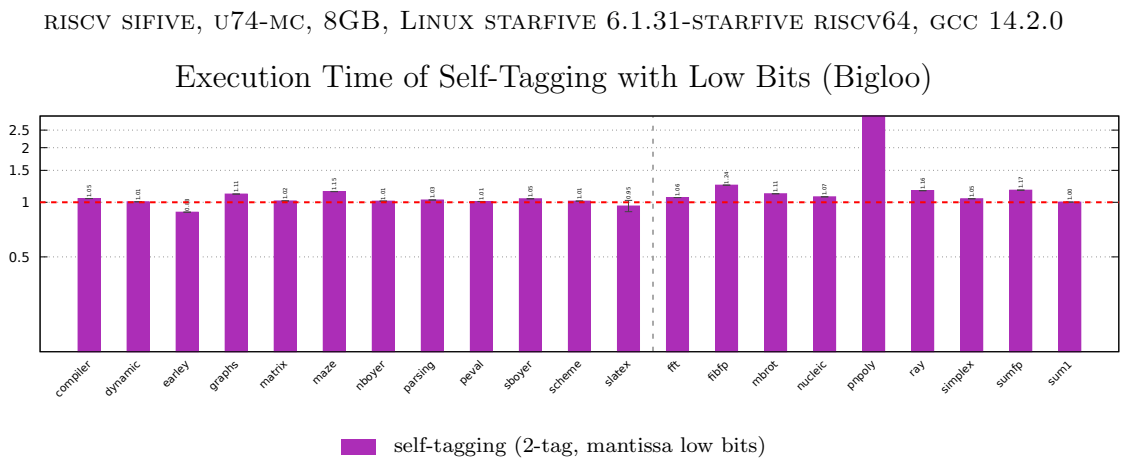


Figure 19: Execution time of Bigloo with self-tagging using the mantissa low bits relative to allocated floats. The y-axis shows execution time relative to the version that uses tagged pointers for all floats (dashed horizontal red line) on a logarithmic scale. 1.00× indicates no change, lower means faster, and higher means slower execution than allocated floats.